

Fortran (autore: Vittorio Albertoni)

Premessa

Un tempo si usava scrivere FORTRAN, visto che i calcolatori per cui è nato conoscevano soltanto lettere maiuscole.

Dal momento che la versione del linguaggio cui mi riferisco in questo manualetto è moderna mi sento autorizzato a scrivere Fortran.

FORTRAN è l'acronimo di FORMula TRANslation (oppure FORMula TRANslator) che significa traduzione (o traduttore) di formule e il primo compilatore FORTRAN fu sviluppato a partire dal 1954 per il calcolatore IBM 704 da un gruppo di lavoro guidato da John Backus.

Prima del FORTRAN un qualsiasi programma per fare calcoli doveva essere scritto con il linguaggio assembly o direttamente in codice macchina: come dire che il comune mortale era costretto a fare i suoi calcoli affidandosi ad un foglio ed a una matita, magari con il supporto di tavole logaritmiche.

La più grande innovazione del FORTRAN è stata l'introduzione nel linguaggio di programmazione degli operatori aritmetici. E' infatti attraverso i simboli $+$, $-$, $*$ e $/$ che diventa possibile rappresentare formule matematiche che vengono poi tradotte in linguaggio macchina da un compilatore.

Quando è nato il linguaggio le istruzioni venivano immesse su scheda perforata e scritte dalla colonna 7 alla colonna 72 della scheda, riservando le colonne da 2 a 5 a etichette utilizzabili per i salti condizionati, la colonna 1 a contrassegnare righe di commento e la colonna 6 ad indicare eventuali continuazioni del codice sulla riga successiva.

Fino alla versione 77 del linguaggio questa formattazione doveva essere rispettata anche utilizzando editor di testo in luogo di perforatori di schede.

E' solo con la versione 90 che il codice Fortran può essere scritto su editor di testo senza rispettare questa formattazione e potendo utilizzare a piacere lettere minuscole e maiuscole.

A questo svecchiamento del linguaggio si è accompagnato il crescente interesse a renderlo utilizzabile su computer più evoluti da quello per il quale fu concepito.

Già nel 1977 la Microsoft rilasciò un compilatore Fortran, al quale lavorò personalmente anche Bill Gates.

Nel mondo del software libero, con la nascita del sistema operativo GNU/Linux, nell'ambito del progetto GNU, la suite di compilazione gcc è stata arricchita dal compilatore gfortran, che porta il linguaggio Fortran sul personal computer.

Questo manualetto si riferisce alla versione 90 del linguaggio Fortran e al compilatore gfortran.

Versione del linguaggio e compilatore non sono i più recenti: già esiste la versione Fortran 2023 e il relativo compilatore predisposto dalla Intel. Mi sono fermato a Fortran 90 in quanto è meno complicato e il suo compilatore è software libero.

Giusto segnalare che, sempre come software libero, esiste un altro compilatore che si chiama lfortran. Ho preferito presentare gfortran in omaggio al progetto GNU di cui fa parte.

L'interesse ancora vivo per il linguaggio Fortran nonostante esistano linguaggi altrettanto votati al calcolo come Python e Julia sta nel fatto che, a differenza dei citati linguaggi, il Fortran, senza essere più difficile da apprendere, è un linguaggio compilato e genera file eseguibili e, rispetto ad altri linguaggi compilati come C e Pascal, rende più agevole trattare calcoli.

Indice

1	Installazione	3
2	Come funziona	3
3	Tipi	3
3.1	Tipi fondamentali	3
3.2	Tipi contenitore	4
4	Variabili e costanti	4
5	Operatori	5
6	Trattamento dei contenitori	6
6.1	Stringhe	6
6.2	Array	7
7	Funzioni intrinseche	7
7.1	Funzioni trigonometriche	7
7.2	Funzioni di potenza e radice:	8
7.3	Funzioni di valore assoluto e parte intera:	8
7.4	Funzioni per la conversione di tipo:	8
8	Interattività con l'utente	8
9	Struttura del programma	10
9.1	Funzioni	10
9.2	Subroutine	10
10	Primi programmini di esempio	11
11	Strutture di controllo	12
11.1	if	12
11.2	case	13
11.3	do	14
11.4	do while	14
12	Altri programmi di esempio	14
13	Lavorare con i file	17
13.1	Scrittura	17
13.2	Lettura	17
14	Considerazioni conclusive	17

1 Installazione

Tutto ciò che c'è da sapere su Fortran si trova all'indirizzo <https://fortran-lang.org/>.

Se apriamo la pagina LEARN troviamo indicazioni sulla documentazione esistente per imparare ed utilizzare il linguaggio.

In particolare troviamo l'accesso ad un tutorial di base cliccando su QUICK START TUTORIAL.

Se nel menu a lato di questa pagina clicchiamo su SETTING UP YOUR OS e poi su INSTALLING GFORTRAN andiamo all'indirizzo https://fortran-lang.org/learn/os_setup/install_gfortran/ dove troviamo istruzioni per installare il compilatore che qui ci interessa.

Il compilatore è disponibile per Linux, Windows e Mac, ma Linux è l'ambiente ideale per utilizzarlo.

Chi usa Linux lo trova sicuramente nel repository della distro e lo può installare con i comandi

```
sudo apt install gfortran
```

o

```
sudo dnf install gcc-gfortran
```

a seconda se, rispettivamente, ci troviamo nel mondo Debian (Debian, Ubuntu e derivate, MXLinux, Linux Mint, ecc.) o RedHat (Fedora, SUSE, ecc.).

Prima di installare è bene passare a terminale il comando

```
gfortran
```

in modo da verificare se per caso non sia già installato e, in caso contrario, ottenere istruzioni per l'installazione.

2 Come funziona

Utilizzando la sintassi descritta in questo manualetto, con un qualsiasi editor di testo si scrive il programma e lo si salva con estensione `.f90`.

Il compilatore che abbiamo installato riconosce anche le versioni 77 e 95 e vi si adegua in base all'estensione del file sorgente (`.f77` o `.f95`), ma in questo manuale trattiamo la versione 90.

La compilazione avviene con il comando a terminale

```
gfortran -o <nome_eseguibile> <sorgente>
```

dove

`<nome_eseguibile>` è il nome da dare al file eseguibile

`<sorgente>` è il file `.f90` contenente le istruzioni del programma.

Utilizzando l'editor Geany possiamo agevolmente scrivere e compilare senza spostarci dall'editor stesso.

3 Tipi

Come tutti i linguaggi di programmazione strutturata Fortran è fortemente tipizzato a tipizzazione statica.

I tipi dei dati elaborabili con questo linguaggio sono i seguenti.

3.1 Tipi fondamentali

Tipi numerici

Sono i seguenti

- `. integer` per i numeri interi,

- `. real` per i numeri decimali a virgola mobile,

- `. complex` per i numeri complessi composti da una parte reale e da una parte immaginaria.

La precisione di default è semplice, a 32 bit, pertanto

- . il massimo numero intero trattabile con segno è dato da $2^{32}/2 - 1 = 2.147.483.647$
- . i numeri in virgola mobile contengono 7 cifre decimali esatte.

Possiamo passare a doppia precisione aggiungendo alla parola chiave che identifica il tipo, tra parentesi tonde, l'indicazione `kind=8`, così

- . `integer(kind=8)` per i numeri interi,
- . `real(kind=8)` per i numeri decimali a virgola mobile,

Con questo la precisione diventa doppia, a 64 bit, pertanto

- . il massimo numero intero trattabile con segno è dato da $2^{64}/2 - 1 = 9.223.372.036.854.775.808$
- . i numeri in virgola mobile contengono 15 cifre decimali esatte.

Occorre fare molta attenzione ai limiti relativi agli interi in quanto il Fortran va in overflow senza manifestarlo, semplicemente fornendo risultati sballati senza che ce ne accorgiamo.

Tipi carattere

Il tipo carattere si identifica con
`character`

Tipo booleano

Il tipo booleano si identifica con
`logical`
e rappresenta i valori booleani `.true.` e `.false.`

3.2 Tipi contenitore

I tipi contenitore sono strutture di dati.

Array

Questo tipo si ottiene aggiungendo alla parola chiave che identifica il tipo di dato da contenere, separata da una virgola, la parola chiave `dimension` con, tra parentesi tonde, il numero di elementi da contenere.

Con

`integer, dimension(3)` si identifica un array composto da 3 numeri interi.

L'array bidimensionale (matrice) si identifica indicando la doppia dimensione.

`real, dimension(3,2)` identifica una matrice di numeri decimali di 3 righe e 2 colonne.

Stringa

Si ottiene aggiungendo alla parola chiave `character`, tra parentesi tonde, il numero dei caratteri.

Con

`character(10)` si identifica una stringa di 10 caratteri.

4 Variabili e costanti

Variabili e costanti corrispondono a posizioni di memoria in cui inseriamo valori per averli a disposizione per le elaborazioni previste nel programma.

Il linguaggio Fortran, per default, permette di creare variabili e costanti destinate a contenere valori numerici nel momento in cui servono, con tipizzazione implicita.

La tipizzazione implicita assegna tipo `integer` alle variabili il cui nome inizia con una delle lettere I, J, K, L, M, N (in maiuscolo o in minuscolo) e tipo `real` in tutti gli altri casi.

Purtroppo il tipo assegnato implicitamente non è modificabile e, se in una variabile il cui nome comincia per una delle lettere I, J, K, L, M, N (in maiuscolo o in minuscolo) inseriamo un valore con cifre decimali, esso verrà memorizzato nella variabile come intero, perdendo la parte decimale.

Ad evitare spiacevoli inconvenienti possano derivare da questo stato di cose è consigliabile che non ci avvaliamo dell'impostazione di default e scegliamo di operare escludendola indicando nella prima riga del programma la dizione

```
implicit none.
```

Fatta questa scelta, per poter usare una variabile o una costante dobbiamo prima averla dichiarata indicando il tipo di dato che essa andrà a contenere.

La variabile contiene un valore modificabile, la costante contiene un valore fisso, che non può essere modificato.

Per dichiarare una variabile usiamo la sintassi

```
<tipo> :: <nome>
```

Possiamo inizializzare la variabile nel momento della sua dichiarazione assegnandole un primo valore con la sintassi

```
<tipo> :: <nome> = <valore>
```

Con

```
integer :: raggio creiamo la variabile raggio destinata a contenere un intero a precisione singola,
```

```
real :: raggio=12.5 creiamo la variabile raggio con tipo real e le assegniamo il valore iniziale 12,5,
```

```
integer(kind=8) :: a creiamo la variabile a destinata a contenere un intero a precisione doppia,  
character(10) :: nome="Pippo" creiamo la variabile nome destinata a contenere una stringa di 10 caratteri e le assegniamo il valore iniziale Pippo.
```

La stringa si scrive tra apici semplici o doppi).

```
integer,dimension(3) :: vettore=[1,2,3] crea un array, denominato vettore, e gli assegna i valori 1, 2 e 3
```

L'array si crea indicandone i valori, separati da virgola, tra parentesi quadre.

```
real,dimension(3,2) :: M crea la variabile M destinata a contenere una matrice di 3 righe e 2 colonne.
```

Nel caso delle matrici gli elementi si inseriscono attraverso gli indici di riga e di colonna.

Nello specifico $M(1,1) = 3.5$ assegna il valore 3,5 al primo elemento della prima riga,

$M(2,2) = 1.75$ assegna il valore 1.75 al secondo elemento della seconda riga, ecc.

Per dichiarare una costante usiamo la sintassi

```
<tipo>, parameter :: <nome> = <valore>
```

con

```
real,parameter :: pi=3.14159265 creiamo la costante pi con assegnato il valore di  $\pi$ .
```

5 Operatori

Gli operatori collegano tra loro operandi di varia natura in espressioni che forniscono un risultato.

Operatori aritmetici

In ordine di esecuzione sono i seguenti:

****** per l'elevamento a potenza

***** per la moltiplicazione

/ per la divisione (per avere un risultato reale almeno uno degli operandi deve essere reale)

+ per la somma

- per la sottrazione

Questi operatori si applicano tra valori numerici e, applicati a vettori e matrici eseguono le operazioni elemento per elemento.

La vera moltiplicazione tra matrici si effettua con il comando
`matmul(<matrice>, <matrice>)`

Operatori di confronto

Servono per confrontare due valori e il risultato che restituiscono è un valore booleano.

Sono i seguenti.

`==` uguale,
`/=` non uguale,
`<` minore,
`<=` minore o uguale,
`>` maggiore,
`>=` maggiore o uguale,

Gli operandi associati al confronto devono avere lo stesso tipo.

Operatori logici

Sono i seguenti.

`.and.` che sta per `and`,
`.or.` che sta per `or`,
`.not.` che sta per `not`.

6 Trattamento dei contenitori

6.1 Stringhe

Come anticipato nel Capitolo 4, la stringa si crea scrivendo i caratteri che la compongono tra apici semplici o doppi.

Per il trattamento delle stringhe esistono alcune funzioni:

`len(<stringa>)` restituisce il numero dei caratteri di una stringa

`len('ciao')` restituisce 4,

Se la stringa è contenuta in una variabile questa funzione, applicata al nome della variabile, restituisce il numero dei caratteri indicati nella dichiarazione della variabile, indipendentemente dal numero dei caratteri che compongono la stringa assegnata.

`trim(<variabile_stringa>)` restituisce la stringa contenuta in una variabile eliminando gli spazi bianchi.

Se abbiamo una variabile stringa `nome`, dichiarata di lunghezza 10, con assegnata la stringa "Luigi",

`len(nome)` restituisce 10

`len(trim(nome))` restituisce 5

La funzione è utile quando si concatenano stringhe per eliminare eventuali spazi bianchi.

`<variabile_stringa>(indice_inizio:indice_fine)` estrae una sottostringa dalla stringa contenuta in una variabile, formata dai caratteri contenuti tra l'indice di inizio e l'indice di fine.

Ricordo che i caratteri della stringa sono indicizzati per default a partire da 1.

Se abbiamo una variabile stringa `nome` con assegnata la stringa "Luigi"

`nome(2:4)` restituisce la sottostringa `uig`

`nome(2:2)` restituisce il carattere `u`

Per concatenare stringhe si usa l'operatore `//` inserendolo tra le stringhe da concatenare.

6.2 Array

Come anticipato nel Capitolo 4, l'array monodimensionale (vettore) si crea scrivendo i dati che lo compongono, separati da virgola, tra parentesi quadre e gli array bidimensionali (matrici) si scrivono inserendo gli elementi attraverso gli indici di riga e di colonna.

Dichiarato l'array A di 4 elementi, con

```
A = [1,2,3,4]
```

vi inseriamo i numeri 1, 2, 3 e 4.

Dichiarato l'array M di 2 righe e 2 colonne, con

```
M(1,1) = 1
```

```
M(1,2) = 2
```

```
M(2,1) = 3
```

```
M(2,2) = 4
```

vi inseriamo la matrice $\begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix}$

Per il trattamento degli array esistono alcune funzioni:

`sum(variabile_array)` calcola la somma di tutti gli elementi di un array,

`product(variabile_array)` calcola il prodotto di tutti gli elementi di un array,

`maxval(variabile_array)` restituisce il valore massimo di un array di numeri reali o interi,

`minval(variabile_array)` restituisce il valore minimo di un array di numeri reali o interi,

che si aggiungono alla funzione `matmul()` per la moltiplicazione di matrici, che abbiamo già indicato nel precedente Capitolo 5.

Per estrarre un singolo elemento da un array si scrive il nome dell'array seguito, tra parentesi tonde, dall'indice dell'elemento da estrarre (nel caso di matrici composto dall'indice di riga e dall'indice di colonna separati da virgola).

Dati gli array A e M degli esempi appena visti,

A(3) ritorna 3

M(2,2) ritorna 4

Nel caso di matrici possiamo estrarre gli elementi che compongono righe e colonne con le sintassi

`<nome_matrice>(<indice_riga>, :)` per estrarre gli elementi della riga indicata

`<nome_matrice>(:, <indice_colonna>)` per estrarre gli elementi della colonna indicata

Data la matrice M degli esempi appena visti

M(1, :) estrae i valori 1 2

M(:, 1) estrae i valori 1 3

7 Funzioni intrinseche

Oltre agli operatori ed alle funzioni viste nel precedente Capitolo 6 relative al trattamento degli array, il linguaggio ci fornisce altre funzioni preconfezionate che possiamo utilizzare per le nostre elaborazioni.

7.1 Funzioni trigonometriche

Con l'argomento x di tipo real abbiamo:

`sin(x)` ritorna il seno di x radianti

`cos(x)` ritorna il coseno di x radianti

`tan(x)` ritorna la tangente di x radianti

`asin(x)` ritorna l'arcoseno di x in radianti

`acos(x)` ritorna l'arcocoseno di x in radianti

`atan(x)` ritorna l'arcotangente di x in radianti

7.2 Funzioni di potenza e radice:

Con l'argomento x di tipo real abbiamo:

`sqrt(x)` per la radice quadrata di x

`exp(x)` per l'esponenziale di $x e^x$

`log(x)` per il logaritmo naturale di x

`log10(x)` per il logaritmo decimale x

7.3 Funzioni di valore assoluto e parte intera:

`abs(x)` per il valore assoluto di x

`int(x)` ritorna la parte intera del numero reale x come numero intero

`nint(x)` ritorna il numero intero più vicino al numero reale x .

7.4 Funzioni per la conversione di tipo:

`real(x)` converte il numero intero x in numero reale

`int(x)` converte il numero reale x in numero intero perdendo la parte decimale

`cmplx(parte reale, parte immaginaria)` converte due numeri in un numero complesso.

8 Interattività con l'utente

Qualsiasi programma per computer deve necessariamente interagire con l'utente, quanto meno per evidenziare all'utente stesso i risultati delle elaborazioni previste nel programma stesso e, spesso, anche per chiedere all'utente di inserire dati o esprimere scelte per proseguire le elaborazioni.

Le funzioni che abbiamo a disposizione per queste cose sono le seguenti.

print*,

E' l'istruzione più semplice per stampare dati o messaggi sul dispositivo di output standard (console).

L'asterisco sta ad indicare che viene usata una formattazione di default. Per esempio che verranno scritti 7 decimali per i tipi a precisione singola e 15 decimali per i tipi a precisione doppia.

La virgola separa il comando da ciò che si deve scrivere e, se dobbiamo scrivere più elementi con lo stesso comando, separa gli elementi da scrivere.

Pertanto il comando `print*` senza la virgola può solo servire a produrre una riga vuota nell'output.

Gli elementi da scrivere, che indichiamo dopo la virgola, possono essere nomi di variabili (e ne verrà scritto il valore), letterali numerici o espressioni numeriche (e ne verrà scritto il risultato), stringhe (parole o frasi racchiuse tra doppi apici).

Esempio:

Supponendo di avere una variabile `a` contenente il valore 12.5, l'istruzione

`print*, "3 moltiplicato 4 fa ", 3*4, " e la variabile a vale ", a`,

scrive a terminale quanto segue

```
3 moltiplicato 4 fa      12  e la variabile a vale      12.5000000
```

Come si vede, la formattazione di default scrive il valore della variabile di tipo real `a` con 7 decimali e inserisce spazi tra le stringhe e i valori.

read*,

E' l'istruzione più semplice per acquisire dati dal dispositivo di input standard (console).

L'asterisco indica che la lettura del dato avviene secondo il formato standard di Fortran.

La virgola separa il comando dall'indicazione di dove collocare ciò che si legge e, se dobbiamo acquisire più elementi con lo stesso comando, separa gli elementi da acquisire.

Prima di questa istruzione è bene inserire nel programma, utilizzando l'istruzione `print*,`, un messaggio per richiedere ciò che occorre inserire.

* * *

Se abbiamo la necessità di avvalerci di dispositivi di input o di output diversi da quello standard o di una formattazione dei dati diversa da quella standard abbiamo a disposizione il modo per indicare dispositivi diversi o formattazione diversa dei dati attraverso l'attribuzione di opportuni parametri alle funzioni di input e output. Per ottenere questa possibilità dobbiamo aggiungere all'istruzione di input o di output questi parametri, tra parentesi tonde, al posto dell'asterisco e della virgola, con la sintassi

`<comando>(<dispositivo>, <direttiva_formattazione>)`.

Per utilizzare dispositivo e formattazione di default, tra le parentesi tonde inseriamo due asterischi separati da virgola.

`read(*,*)` equivale a `read*,`.

Quanto al dispositivo, pertanto, se lavoriamo con la tastiera e lo schermo continuiamo ad usare l'asterisco. Vedremo cosa fare quando parleremo di leggere e scrivere dati su file.

Quanto alla formattazione lasciamo ai professionisti occuparsi della formattazione in lettura e qui vediamo come formattare ciò che scriviamo utilizzando la direttiva di formattazione, che è una stringa contenente, tra parentesi tonde, indicazioni per come formattare ciascuno degli elementi da scrivere (che, se più di uno, sono separati da virgola).

Le principali indicazioni che utilizziamo sono:

`an` che indica di scrivere i primi `n` caratteri di una stringa. Se omettiamo `n` la stringa viene scritta per la lunghezza dichiarata lasciando spazi corrispondenti alla lunghezza non utilizzata ¹.

`iw` per gli interi, dove `w` indica il numero di caratteri di ampiezza dello spazio in cui viene scritto il numero allineato a destra (pari a 12 nella formattazione standard).

`fw.d` per i reali, dove `w` indica il numero di caratteri di ampiezza dello spazio in cui viene scritto il numero allineato a destra e `d` indica il numero di cifre decimali da esporre (se `w` è 0 viene occupato lo spazio che serve per scrivere il numero senza allineare a destra).

`nx` inserisce `n` spazi bianchi.

* * *

write()

E' l'istruzione per stampare dati o messaggi che può utilizzare indicazioni diverse dallo standard in merito a dispositivo di output e di formattazione, indicazioni che si passano come argomenti tra le parentesi.

Ovviamente, per quanto già detto,

`write(*,*)` equivale a `print*,`.

Tenuto conto di quanto detto, riscriviamo l'esempio che abbiamo visto parlando dell'istruzione `print*`, sempre supponendo di avere una variabile `a` contenente il valore 12.5, con le direttive di formattazione così

`write(*, "(a,i0,a,f0.2)")` "3 moltiplicato 4 fa ", 3*4, " e la variabile a vale ", `a` ottenendo

```
3 moltiplicato 4 fa 12 e la variabile a vale 12.50
```

che è un po' meglio di prima.

¹Senza utilizzare questa indicazione di formattazione possiamo eliminare gli spazi bianchi utilizzando la funzione `trim` che abbiamo visto nel Paragrafo 6.1.

9 Struttura del programma

Un programma in linguaggio Fortran 90 inizia con la riga
`program <nome_programma>`, che attribuisce un nome al programma,
e termina con la riga
`end program <nome_programma>`

Tra queste due righe si scrivono le istruzioni, sapendo che il linguaggio ignora maiuscole e minuscole, normalmente utilizzando una riga per ogni istruzione.

E' possibile scrivere più istruzioni sulla stessa riga separandole con il punto e virgola (;), ma si tratta di una pratica che peggiora la leggibilità del programma.

Eventuali commenti sono inseribili dopo il carattere punto esclamativo (!).

Subito dopo la prima riga, se optiamo per la programmazione strutturata, inseriamo l'istruzione

```
implicit none
```

A seguire inseriamo le dichiarazioni delle variabili che intendiamo utilizzare e dei relativi tipi secondo la sintassi vista nel Capitolo 4.

Alla sezione delle dichiarazioni segue la sezione delle istruzioni da eseguire.

Le istruzioni da eseguire possono essere raggruppate in funzioni e subroutine, quando ciò è funzionale all'economia del programma. Per esempio per evitare di riscrivere più volte nello stesso programma una serie di istruzioni ricorrenti.

Per poter includere il codice di funzioni e subroutine nel programma occorre far precedere alle relative istruzioni la parola chiave

```
contains
```

9.1 Funzioni

Sono destinate a restituire un valore su chiamata e passaggio di parametri.

Il codice di una funzione inizia con la riga

```
<tipo> function <nome>(<argomenti>)
```

e termina con la riga

```
end function <nome>
```

Con <tipo> si indica il tipo di dato restituito dalla funzione.

Se gli argomenti sono più di uno si inseriscono separati da virgola (,).

Il valore che la funzione deve restituire viene assegnato ad una variabile che ha lo stesso nome della funzione.

La funzione si richiama con il suo nome seguito dagli argomenti richiesti tra parentesi tonde (se più di uno separati da virgola) e il valore restituito può essere assegnato ad una variabile o ad una istruzione di stampa.

9.2 Subroutine

Sono destinate a compiere un'azione senza restituire valori.

Il codice di una subroutine inizia con la riga

```
subroutine <nome>
```

e termina con la riga

```
end subroutine <nome>
```

La subroutine si richiama con la parola chiave `call`.

Il seguente codice esemplifica ciò che abbiamo detto

```
program esempio
print*, "Esempio di programma che contiene subroutine e funzioni"
call saluta
print*, somma(23,8)
contains
```

```

subroutine saluta
print*, "Ciao, Vittorio!"
end subroutine saluta
integer function somma(a,b)
integer :: a, b
somma = a+b
end function somma
end program esempio

```

L'esecuzione di questo programma porta il seguente risultato

```

Esempio di programma che contiene subroutine e funzioni
Ciao, Vittorio!
31

```

10 Primi programmini di esempio

Tanto per mostrare che Fortran 90 può anche fare cose diverse dai calcoli cominciamo da un programmino che richiede il nome all'utente per salutarlo

```

program saluto
character(10) :: nome
print*, "Come ti chiami?"
read*, nome
print*, "Ciao, ", trim(nome), "!"
end program saluto

```

Notare l'uso della funzione trim() per stampare il nome. Se non usassimo questo accorgimento il punto esclamativo si collocherebbe in stampa non alla fine del nome ma alla fine dei 10 caratteri riservati alla relativa variabile.

Quest'altro programma mostra i valori della circonferenza e dell'area di un cerchio il cui raggio è indicato dall'utente

```

program cerchio
implicit none
real :: raggio, circonferenza, area
real,parameter :: pi = acos(-1.0)
print*, "Inserisci il raggio del cerchio"
read*, raggio
circonferenza = 2*pi*raggio
area = raggio**2*pi
print*, "per un cerchio avente il raggio indicato"
write(*,"(a,f15.3)") "la circonferenza è ", circonferenza
write(*,"(a,f15.3)") "l'area è          ", area
end program cerchio

```

Notare la creazione della costante per un valore approssimato di π attraverso l'utilizzo di una funzione trigonometrica e la formattazione della stampa dei risultati delle elaborazioni.

La schermata di esecuzione di questo programma è la seguente

```

Inserisci il raggio del cerchio
24.75
per un cerchio avente il raggio indicato
la circonferenza è          155.509
l'area è                    1924.422

```

11 Strutture di controllo

Come tutti i linguaggi di programmazione, Fortran ci offre il modo di controllare l'esecuzione del programma attraverso istruzioni per condizionare l'esecuzione di un blocco al verificarsi di certe condizioni oppure per la ripetizione di blocchi.

11.1 if

L'istruzione `if`, chiamata istruzione di esecuzione condizionale (in inglese `if` è il nostro `se`), ci dà modo di assoggettare l'esecuzione di un blocco di istruzioni al verificarsi di una determinata condizione: se la condizione è vera viene eseguito il blocco di istruzioni, altrimenti si prosegue l'esecuzione del programma saltando il blocco stesso.

La sintassi è la seguente

```
if (<condizione>) then
  <istruzioni>
end if
```

dove <condizione> è una qualsiasi espressione che relaziona due valori attraverso operatori di confronto: se la condizione si verifica vengono eseguite la o le istruzioni indicate, altrimenti si passa oltre.

L'istruzione `if` si presta anche all'esecuzione condizionale a due rami. Per ottenere questo dobbiamo abbinarla all'istruzione `else` con questa sintassi

```
if (<condizione>) then
  <istruzioni>
else
  <istruzioni>
end if
```

In questo caso se la condizione si verifica vengono eseguite le istruzioni contenute nel primo blocco altrimenti vengono eseguite quelle contenute nel secondo blocco dopo `else` (altrimenti). In ogni caso proseguendo poi nell'esecuzione del resto del programma.

Possiamo infine gestire l'esecuzione condizionale a più rami abbinando a `if` l'istruzione `else if` con questa sintassi

```
if (<condizione>) then
  <istruzioni>
else if (<condizione>) then
  <istruzioni>
else if (<condizione>) then
  <istruzioni>
.....
else
  <istruzioni>
end if
```

Il seguente programma

```
program esempio_if_1
integer :: n
print*, "Inserisci un numero"
read*, n
if (n<10) then
print*, "Sei dentro"
else
print*, "Ciao"
end if
end program esempio_if_1
```

se inseriamo un numero minore di 10 stampa la frase «Sei dentro»; se inseriamo 10 o un numero superiore stampa il saluto «Ciao».

Quest'altro

```
program esempio_if_2
integer :: n
print*, "Inserisci un numero"
read*, n
if (n<10) then
print*, "Sei dentro"
else if (n>=10 .and. n<=12) then
print*, "Sei appena fuori"
else
print*, "Sei completamente fuori"
end if
print*, "Ciao"
end program esempio_if_2
```

se inseriamo un numero minore di 10 stampa la frase «Sei dentro» e poi il saluto «Ciao»; se inseriamo 10 o 11 o 12 stampa la frase «Sei appena fuori» e poi il saluto «Ciao»; se inseriamo un numero superiore a 12 stampa la frase «Sei completamente fuori» e poi il saluto «Ciao».

11.2 case

Il costrutto case è un altro modo di fare ciò che possiamo fare con la serie di else if e else.

Con questo costrutto l'ultimo esempio che ho fatto può essere scritto così

```
program case_1
integer :: n
print*, "Inserisci un numero"
read*, n
select case (n)
case (1:9)
print*, "Sei dentro"
case (10:12)
print*, "Sei appena fuori"
case default
print*, "Sei completamente fuori"
end select
print*, "Ciao"
end program case_1
```

Come si vede, si abbinano direttamente istruzioni semplici ai valori che può assumere una determinata variabile, che può anche essere una stringa, come vediamo nel seguente esempio.

```
program case_2
character(6) :: colore
print*, "Indica il colore del semaforo in lettere minuscole"
read*, colore
select case (colore)
case("rosso")
print*, "Fermati"
case("verde")
print*, "Passa tranquillo"
case("giallo")
print*, "Rallenta perché dovrai fermarti"
case default
print*, "Dovevi inserire uno dei colori del semaforo in minuscolo"
end select
end program case_2
```

In questo caso, a seconda del colore che scriviamo, abbiamo la risposta che ci indica come comportarci e se inseriamo una parola che non indica un colore del semaforo o se non usiamo l'iniziale minuscola come richiesto abbiamo l'invito a comportarci come si deve.

11.3 do

Si usa per ripetere l'esecuzione di una istruzione o di un blocco di istruzioni per un numero definito di volte.

La sintassi per l'uso di questa istruzione è

```
do contatore = <inizio>, <fine>
<istruzioni>
end do
```

La variabile che ho battezzato contatore per richiamarne la natura (in genere la si indica semplicemente con la lettera i), deve essere prima dichiarata di tipo intero e con <inizio> e <fine> si indicano i valori del contatore per l'inizio del ciclo e per la fine del ciclo.

Questo piccolo programma

```
program do
integer :: i
do i = 1, 5
print*, "Ciao"
end do
end program do
```

scrive la parola Ciao per cinque volte

Quest'altro

```
program do
integer :: i
do i = 1, 10
print*, i
end do
end program do
```

elenca i numeri da 1 a 10.

11.4 do while

Si usa per ripetere l'esecuzione di una istruzione o di un blocco di istruzioni finché una specifica condizione rimane vera.

La sintassi per questa istruzione è

```
do while (<condizione>)
<istruzioni>
end do
```

Per scrivere cinque volte la parola Ciao utilizzando questo costrutto possiamo procedere così

```
program do_while
integer :: i = 0
do while (i < 5)
print*, "Ciao"
i = i+1
end do
end program do_while
```

12 Altri programmi di esempio

Con le strutture di controllo abbiamo esaurito l'esame degli strumenti che ci mette a disposizione il linguaggio Fortran 90 per creare eseguibili da riga di comando, cioè senza interfaccia grafica e possiamo fare qualche esempio un tantino più complicato di quelli presentati nel Capitolo 10.

Questo programma elenca i numeri primi contenuti in un certo intervallo.

```

program primi
  implicit none
  integer :: inizio, fine, i
  print*, "RICERCA DEI NUMERI PRIMI IN UN INTERVALLO"
  print*, "Inserisci il numero di inizio dell'intervallo"
  read*, inizio
  print*, "Inserisci il numero di fine dell'intervallo"
  read*, fine
  do i = inizio, fine
    call sePrimo(i)
  end do
contains
subroutine sePrimo(n)
  implicit none
  integer :: n, k
  logical :: e_primo
  e_primo = .true.
  if (n <= 1) then
    e_primo = .false.
  else if (n <= 3) then
    e_primo = .true.
  else
    do k = 2, int(sqrt(real(n)))
      if(mod(n, k) == 0) then
        e_primo = .false.
        exit
      end if
    end do
  end if
  if (e_primo) then
    print*, n
  end if
end subroutine sePrimo
end program primi

```

Le indentazioni non sono necessarie e le ho utilizzate per rendere più leggibile il programma ².

Il programma definisce la subroutine `sePrimo()` che, secondo un semplice algoritmo che non ho inventato io ma ho trovato in rete, verifica se un numero è un numero primo e, se sì, lo stampa.

All'utente vengono chiesti gli estremi dell'intervallo di ricerca e la subroutine `sePrimo()` viene applicata ai numeri contenuti nell'intervallo.

Notare l'uso dell'istruzione `exit` per interrompere, se e quando necessario, un ciclo.

Quest'altro programma calcola il fattoriale di un numero (corrispondente al prodotto di quel numero per il prodotto di tutti i numeri interi che lo precedono) applicando la precisione a 64 bit in modo da ottenere e scrivere anche numeri molto grandi, fino a 19 cifre.

Purtroppo, anche con questo accorgimento, non andiamo oltre il calcolo del fattoriale di 20, corrispondente a 2.432.902.008.176.640.000 (il fattoriale di 21 impegna 20 cifre e genera stack overflow evidenziando un risultato errato).

²Se si usa l'editor Geany e si vogliono inserire indentazioni occorre avere l'avvertenza di crearle con la barra spaziatrice. Se si vogliono creare con il tasto di tabulazione occorre preventivamente intervenire da menu MODIFICA ► PREFERENZE ► EDITOR e, nella scheda INDENTAZIONE selezionare l'opzione SPAZI. Ciò in quanto il compilatore Fortran 90 non riconosce il carattere di tabulazione.

```

program fattoriale
implicit none
integer :: n, i
integer(kind=8) :: f
print*, 'Inserire il valore di cui si vuole il fattoriale '
read*, n
i = 1
f = 1
100 if (i>n) then
    write(*,"(a,i0,a,i20)") 'Il fattoriale di ', n , ' è', f
    stop
else
    f = f * i
    i = i + 1
end if
go to 100
end program fattoriale

```

Notare come il ciclo per calcolare il fattoriale sia stato scritto utilizzando la tecnica del GO TO che dirotta l'esecuzione verso una etichetta: nel caso l'etichetta 100 che contiene la verifica che il contatore i non sia superiore al numero n di cui vogliamo calcolare il fattoriale. Inizializzate a 1 le variabili i e n, nel caso i sia superiore a n si stampa il risultato e si interrompe il ciclo. Fino a quando ciò non avviene (else) si calcola il prodotto tra il valore di i e il prodotto precedente (f) aumentando di 1 il valore di i.

Ora un programma per trovare le radici di una equazione di secondo grado

```

program equazione
implicit none
real :: a, b, c
real :: delta
real :: x1, x2
real :: i, r
print*, "RISOLUZIONE DELL'EQUAZIONE DI SECONDO GRADO"
print*, "Introdurre i coefficienti a, b e c, separati da virgola"
read*, a, b, c
print*, 'I coefficienti immessi sono: ', a, b, c
delta = b**2-4*a*c
if(delta>0) then
    x1 = ( -b + sqrt(delta) ) / ( 2. * a )
    x2 = ( -b - sqrt(delta) ) / ( 2. * a )
    print*, "L'equazione ammette le due radici reali distinte:"
    print*, 'X1 = ', x1
    print*, 'X2 = ', x2
else if(delta<0) then
    r = (-b)/(2.*a)
    i = sqrt(abs(delta))/(2.*a)
    print*, "L'equazione ammette due radici complesse coniugate:"
    print*, 'X1 = ', r, ' + ', i, 'i'
    print*, 'X2 = ', r, ' - ', i, 'i'
else if(delta==0) then
    x1 = (-b)/(2.*a)
    print*, 'Questa equazione ammette la sola radice, doppia:'
    print*, 'X1 = X2 = ', x1
end if
end program equazione

```


13 Lavorare con i file

Per operare su file dobbiamo innanzi tutto assegnare il file ad una unità logica con l'istruzione `open(unit=<numero_intero>, file=<percorso_al_file>)` dove `<percorso_al_file>` indica non solo il nome del file ma tutto il percorso per dove esso si trova nel file system.

Con questa istruzione, se il file non esiste viene creato nel percorso indicato.

Una volta indicato ciò che dobbiamo fare sul file, lo rendiamo operante chiudendo la `unit` con l'istruzione `close(<numero_unit>)`.

13.1 Scrittura

Per scrivere nel file usiamo l'istruzione

```
write(<numero_unit>, <direttiva_formattazione>)
```

Se indichiamo `<direttiva_formattazione>` con un asterisco utilizziamo lo standard.

Questa istruzione scrive sul file corrispondente al numero di `unit` e, se questo esiste già, lo riscrive completamente.

Se intendiamo evitare ciò e aggiungere altri elementi al file senza riscriverlo dobbiamo scrivere l'istruzione di scrittura così

```
write(<numero_unit>, <direttiva_formattazione>, position="append")
```

e il o i nuovi elementi separati da virgola indicati vengono inseriti nella prima riga libera del file, conservando ciò che c'era prima.

13.2 Lettura

Per leggere dati da file, acquisendoli per elaborazioni, dobbiamo preventivamente dichiarare tante variabili quanti sono i dati da leggere.

Poi con l'istruzione

```
read(<numero_unit>, <direttiva_formattazione>) <variabile>, <variabile>,...
```

leggere riga per riga il file assegnando i dati contenuti su ciascuna riga ad altrettante variabili.

Tanto basta per comprendere come acquisire per l'elaborazione un dataset soltanto di qualche centinaio di dati sia un'impresa titanica.

Tra l'altro la cosa si complica per il fatto che il linguaggio non fornisce supporto diretto a formati come csv.

Occorre pertanto risolvere il problema utilizzando librerie esterne ed affrontando comunque notevoli complessità, non alla portata dei dilettanti cui mi rivolgo.

14 Considerazioni conclusive

Ciò che ho esposto in questo manualletto è la base del linguaggio, facilmente comprensibile e memorizzabile, che serve giusto per fare senza fatica programmi di una certa banalità.

Non ho affrontato aspetti di più difficile apprendimento, come l'uso dei puntatori, e non mi sono occupato del possibile utilizzo delle librerie disponibili per rendere facili cose che con le sole funzioni di base del linguaggio sarebbe estremamente complicato realizzare, come l'appena accennata acquisizione di dataset esterni, il calcolo matriciale e l'algebra lineare.

Si sappia, comunque, che queste cose ci sono, anche se non altrettanto numerose, accessibili, facili da usare e documentate di ciò che troviamo a disposizione di altri linguaggi votati al calcolo, come Python e Julia.

Va comunque riconosciuto che Fortran ci dà la possibilità di compilare i nostri programmi e di regalarli ai nostri amici che li possono usare anche senza avere Fortran sul computer.

Programmi che ci forniscono i risultati con velocità neanche lontanamente paragonabile a quella dei linguaggi interpretati, come i cennati Python e Julia e che forse è anche meglio di quella di altri linguaggi compilati, come C e Pascal.